Multi-Format RAG Document Assistant

Overview

In this foundational project, you will develop a RAG application that enables users to upload multiple document types and engage in a conversational interface with that content.

Objective

The application must empower users to:

- Upload one or more documents simultaneously.
- Query the documents using natural language.
- Receive accurate answers derived strictly from the uploaded content to ensure transparency.
- View specific citations indicating exactly where the information was retrieved from within the source files.

Functional Requirements

1. Document Upload & Processing

The system should support PDF, DOCX, TXT, and Markdown (.md) formats. For each upload, the application must:

- **Validate** the file type and extract text efficiently.
- **Clean and Chunk:** Utilize text splitting strategies like `RecursiveCharacterTextSplitter` to manage chunk size and overlap, ensuring context is preserved across segments.
- **Embedding Generation:** Generate high-quality vector representations (e.g., using OpenAI `text-embedding-3-small` or open-source alternatives like Gemma via Ollama).

- **Vector Storage:** Persist chunks and metadata (Document name, Chunk ID, Page number) in a vector database such as ChromaDB, Pinecone, or FAISS.

2. Retrieval & Chat Interface

The chat interface should handle natural language queries (e.g., "Summarize this document" or "Explain the termination clause").

- **Semantic Search:** Generate an embedding for the user query and retrieve the most relevant chunks based on distance metrics like cosine similarity.
- **Grounded Generation:** Pass the retrieved context to the LLM with a prompt ensuring the model only uses the provided data. If information is missing, the model should state it cannot find the answer rather than hallucinating.
- **Citation Layer:** Every response must include a reference to the source document and page number to maintain document lineage and trust.

Suggested Tech Stack

You are encouraged to use industry-standard tools often sought after in AI engineering roles:

Component	Recommended Tools
Language	Python
Framework	FastAPI / LangChain
LLM	OpenAI GPT-4, Gemini, Openrouter or Ollama (Local)
Vector DB	Chroma, Pinecone, or pgvector
Embeddings	OpenAI or Hugging Face Transformers

Evaluation Criteria

Submissions will be assessed based on the following:

- **Technical Implementation:** Correctness of the RAG pipeline and retrieval accuracy.
- **Code Quality:** Clean, well-structured, and documented Python code.
- **Robustness:** Handling irrelevant queries and preventing hallucinations through proper retrieval grading.
- **Documentation:** A clear README and basic API documentation for deployment.

Bonus Features (Optional)

To further demonstrate expertise, consider implementing:

- **Chat History:** Maintaining conversation state for multi-turn dialogues.
- **Hybrid Search:** Combining semantic vector search with keyword-based BM25 retrieval for better accuracy on technical terms.
- **Advanced Patterns:** Implementing a corrective layer (CRAG) to grade retrieved documents before generation.
- **Docker Support:** Containerizing the application for production-ready deployment.